PHASE 1 – REQUIREMENT ANALYSIS INTRODUCTION

Project Title: EduGenie – Google Gemini Powered Learning Assistant

Requirement Analysis is the process of identifying what the EduGenie system should do and what resources are required to develop and operate it.

Purpose of Analysis: To clearly define the needs of students, application functions, technical resources, performance expectations, and security considerations before implementation.

Project Goal: Develop an interactive AI-powered learning assistant that accepts educational questions and provides useful AI-generated explanations.

Expected Outcome: A clear and organized set of requirements that can guide the design, development, testing, and demonstration of EduGenie.

Evidence Placeholder: Insert requirement-analysis discussion or planning screenshot here.

PHASE 2 – USER REQUIREMENTS

Primary Users: Students and learners who need quick explanations and interactive learning assistance.

User Requirements:

• User should be able to open the EduGenie application easily.

• User should be able to enter an educational question.

• User should be able to submit the question.

• User should receive an understandable response.

• User should be able to ask multiple questions during a session.

• User should be able to read the generated response clearly.

• The interface should be simple enough for users with basic computer or mobile skills.

User Expectation: The system should provide a convenient way to obtain learning assistance without a complicated workflow.

Evidence Placeholder: Insert user-requirement notes or planning screenshot here.

PHASE 3 – FUNCTIONAL REQUIREMENTS

Functional requirements describe the specific functions that EduGenie must perform.

FR-01: User Input – The system must accept a question or learning prompt from the user.

FR-02: Input Processing – The application must process the submitted question before sending the request.

FR-03: Gemini Integration – The application must communicate with the configured Google Gemini/Generative AI service.

FR-04: Response Generation – The system must receive the generated response from the AI service.

FR-05: Response Display – The application must display the generated answer to the user.

FR-06: Multiple Queries – The user should be able to submit additional questions.

FR-07: Error Handling – The application should provide an appropriate message when a request cannot be processed.

Evidence Placeholder: Insert functional-requirement table or application workflow screenshot here.

PHASE 4 – NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality and operational characteristics expected from the system.

Usability: The interface should be simple, readable, and easy to navigate.

Performance: The application should process normal requests without unnecessary delay, subject to network and AI-service conditions.

Reliability: The application should handle normal user requests consistently.

Maintainability: The project should use organized code and clear modules so that future changes can be made easily.

Scalability: The design should allow additional learning features to be added later.

Compatibility: The application should work in the supported development/runtime environment and compatible browser where applicable.

Availability: Operation may depend on internet connectivity and availability of the configured AI service.

Evidence Placeholder: Insert non-functional requirement notes or design screenshot here.

PHASE 5 – HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements:

• Computer or laptop for development and testing.

• Basic processor and sufficient RAM for the development environment.

• Stable internet connection for AI-service communication.

• Keyboard and display for entering and viewing content.

Software Requirements:

• Suitable operating system.

• Programming language/environment used for the project.

• Required libraries and dependencies.

• Google Gemini/Generative AI service or configured API access.

• Code editor or IDE.

• Web browser if the project uses a web interface.

Note: Exact versions should be recorded according to the environment actually used by the team.

Evidence Placeholder: Insert development-environment screenshot here.

PHASE 6 – SYSTEM & DATA REQUIREMENTS

System Input: The main input is a text-based educational question or learning prompt.

Processing Requirement: The application prepares the user request and sends it to the configured Gemini model/service.

Output Requirement: The system displays the generated response in a readable format.

Data Requirements:

• User question/prompt.

• Prompt instructions required for educational responses.

• Generated response.

• Any project-specific configuration required for the application.

Data Flow: User Question → Application → Gemini Service → Generated Response → Application Display.

Data Validation: The application may check for empty or invalid input before sending a request.

Evidence Placeholder: Insert data-flow diagram or input/output screenshot here.

PHASE 7 – SECURITY, PRIVACY & ERROR REQUIREMENTS

Security Requirements:

• API keys, passwords, tokens, and credentials must not be exposed in screenshots or public source code.

• Sensitive configuration should be stored using an appropriate secure method such as environment variables.

• Access to project credentials should be limited to authorized team members.

Privacy Consideration: Users should avoid entering sensitive personal information into the learning assistant.

Error Handling Requirements:

• Show a suitable message when the input is empty.

• Handle network or service errors appropriately.

• Avoid exposing technical secrets in error messages.

Testing Requirement: Common valid and invalid inputs should be tested before final demonstration.

Evidence Placeholder: Insert security/configuration or error-handling screenshot here.

PHASE 8 – REQUIREMENT SUMMARY & ACCEPTANCE CRITERIA

Requirement Summary: EduGenie requires a simple user interface, question-input capability, Gemini integration, response generation and display, basic validation, error handling, secure credential management, and a suitable development environment.

Acceptance Criteria:

1. Application opens successfully.

2. User can enter an educational question.

3. Submitted question is processed successfully under normal conditions.

4. Gemini-generated response is received and displayed.

5. User can submit another question.

6. Basic errors are handled appropriately.

7. Credentials are kept private.

8. The final workflow can be demonstrated successfully.

Conclusion: The Requirement Analysis Phase establishes a clear technical and user-focused foundation for developing EduGenie. These requirements can be used as a reference during design, implementation, testing, and demonstration.

Final Evidence Placeholder: Insert completed requirement checklist or team approval screenshot here.